5

Employing functions

This chapter demonstrates how to create reusable blocks of code in functions.

Defining functions

PHP provides many built-in functions that can be called by name in your scripts to execute predefined actions, such as the array sorting functions described here. Additionally, you may create your own custom functions to execute specified actions when called. This usefully allows PHP scripts to be modularized into sections of code that are easier to understand and maintain.

Your custom function names may contain any combination of letters, numbers and the underscore character, but must begin with a letter or an underscore. Understandably, to avoid conflict, the name may not duplicate an existing built-in function name so, for example, you may not create a custom function named “sort”.

Unlike PHP variables, function names are case-insensitive so **cube()**, **Cube()** and **CUBE()** can all reference the same function. Nonetheless, it is recommended you always use the same case as that used to name the function in the function definition.

PHP variable names are case-sensitive, but PHP function names are not.

A function definition is created in a PHP script simply by stating its name after the **function** keyword, followed by parentheses and curly brackets (braces) containing the statements to be executed each time that function gets called:

```
function function_name ( )
{
  statement-to-be-executed ;
  statement-to-be-executed ;
  statement-to-be-executed ;
}
```

A PHP script can then call upon a defined function by name using ***function_name*( );** to execute the statements it contains whenever required. Although the interpreter runs the script from top to bottom, a function call can appear before the function definition. This is not good programming practice, however, so you should define functions before they are called.

Functions may contain any valid PHP script code and may even contain a further function definition. In that case, the inner function will not exist until the outer function has been called.

1. Begin a PHP script with a statement to define a function

```
<?php
function greet()
{
  echo 'Hello User!<br>' ;
}
```

function.php

2. Next, add a statement to call the function defined in the previous step, to have it execute its statement

```
greet() ;
```

3. Now, define another function that contains an inner function definition and a further statement

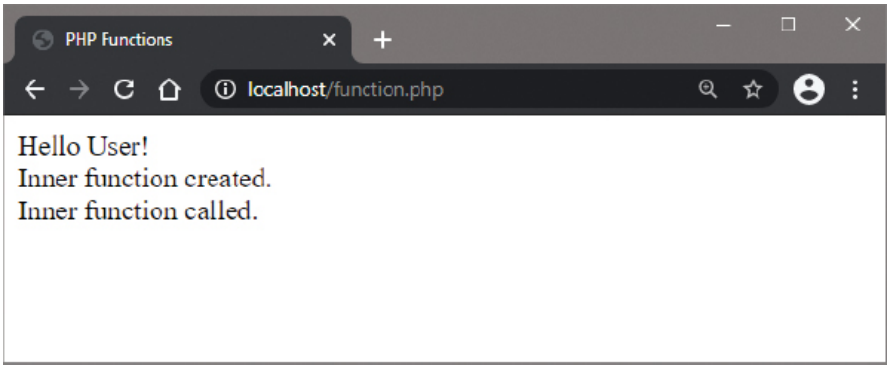
```
function outer()
{
  function inner()
  {
    echo 'Inner function called.<br>' ;
  }
  echo 'Inner function created.<br>' ;
}
```

4. Finally, add statements to call each of the functions defined in the previous step

```
outer() ;
```

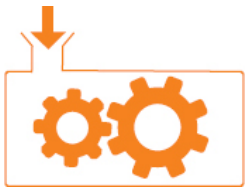
```
inner() ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **function.php** then open the page via HTTP to see the output from each function call



Remove the call to the outer function to see an undefined function error when the script attempts to call the inner function.

Passing arguments



Data can be passed to functions as “arguments” in the same way you can pass arguments to some built-in PHP functions. For example, the **sort()** function takes the array to be sorted as its argument. Functions can be created with any number of arguments but when called, data must be supplied for each argument to avoid an error.

A function accepting arguments is defined by including a comma-separated list of “parameters” within the parentheses after the function name, like this:

```
function function_name ( $arg_1 , $arg_2 , $arg_3 )
{
  statement-to-be-executed ;
  statement-to-be-executed ;
  statement-to-be-executed ;
}
```

Parameter names must adhere to the same naming conventions required for PHP variables and should ideally be meaningful, to represent the nature of the data that will be passed. The argument data must be passed by the call in the correct order and can then be used by the statements within that function.

Parameters must be specified in the function declaration as a comma-separated list, and argument values passed by the caller as a comma-separated list.

Optionally, the function definition can specify the type of data to be passed from the caller by including a type description before the parameter name. Valid type descriptions include **int**, **float**, **string**, **bool**, **callable** and **array**. When types are specified, a call that attempts to pass an incorrect data type to the function will result in an error.

It is important to recognize that by default, argument data is passed “by value”. This means that if a function call specifies a variable name, only a copy of the data within that variable gets passed to the function. Modifying the argument will not affect the original data stored in the variable when it is passed by value.

If you want a function to modify data stored in a variable, specified in a function call, you must pass it “by reference”. This allows the function to operate directly on the original data. Modifying the argument will affect the original data in the variable when it is passed by reference. To ensure that an argument is always passed to a function by reference, you need simply prefix an **&** ampersand before the parameter name in the function definition.

1 Begin a PHP script with a statement to initialize two variables with the same integer value

```
<?php
$a = $b = 5 ;
```

arguments.php

2 Next, define a function that accepts two integer arguments – one by value and the other by reference

```
function modify( int $val , int &$ref )
{
  // Statements to be inserted here.
}
```

3 Now, insert statements to output the passed values, then increment those values and display the modified values

```
echo “Passed values : $val , $ref<br>” ;
$val++ ;
$ref++ ;
echo “Incremented values : $val , $ref<hr>” ;
```

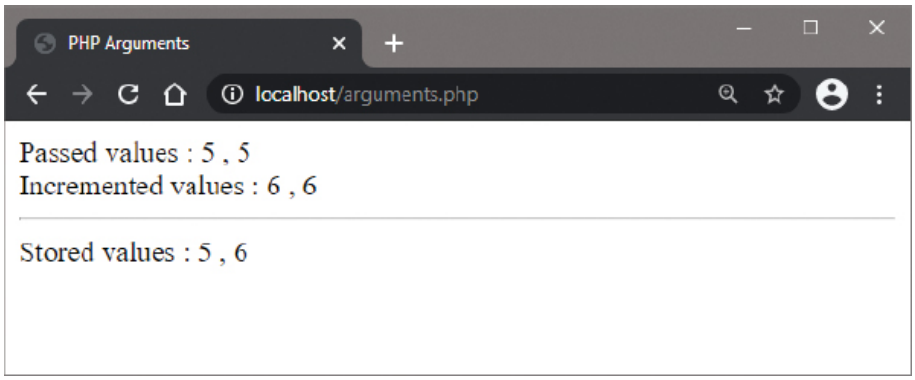
4 After the function block, add a statement to call the function, passing the variable values as arguments

```
modify( $a , $b ) ;
```

5 Finally, add a statement to display the values now stored in each variable

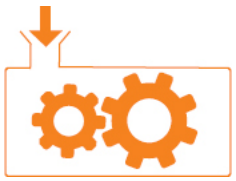
```
echo "Stored values : $a , $b" ;
?>
```

6 Save the document in your web server's **/htdocs** directory as **arguments.php** then open the page via HTTP to see the argument and variable values



The **int** type description in this example specifies that only integer data values can be passed as arguments to the function.

Varying parameters



Default argument values can optionally be specified in a function definition by assignment to the parameter name, like this:

```
function function_name ( $arg_1 , $arg_2 = "Default String" )
{
    statement-to-be-executed ;
    statement-to-be-executed ;
    statement-to-be-executed ;
}
```

This allows a function call to be made that specifies either one or two argument values. If only one argument is specified, it is assigned to the first parameter and the default value is used by the second parameter. Where default values are specified for parameters, they must only appear in the parameter list following (to the right or) parameters that have no specified default value. Otherwise, a call without the correct total number of arguments would cause an error.

If multiple default values are specified there is currently no way to skip a parameter in a function call, to assign an argument value to a particular parameter.

If you need to create a function that will accept an unknown number of arguments, PHP has a special **…** "splat" operator for that purpose. This can be used in a function definition, like this:

```
function function_name ( …$args )
{
    statement-to-be-executed ;
    statement-to-be-executed ;
```

```
    statement-to-be-executed ;
}
```

When the **…** splat operator is used to prefix a parameter name in a function definition, multiple arguments can be passed to that parameter to create an array. The array can be traversed like any other array to manipulate the passed in argument values.

The … splat operator is also sometimes known as the “scatter” operator.

Additionally, the **…** splat operator can be used to “unpack” an array specified in a function call to pass its element values as individual arguments, like this:

```
function function_name ( $arg_1 , $arg_2 , $arg_3 )
{
  echo $arg_1 + $arg_2 + $arg_3 ;
}

$arr = [ 1 , 2 , 3 ] ;

function_name ( …$arr ) ;          # Outputs 6
```

1. Begin a PHP script by defining a function that provides default string values to its two parameters

```
<?php
function drink( string $tmp = ‘hot’ , string $flavor = ‘tea’ )
{
  // Statement to be inserted here.
}
```

parameters.php

3 Next, insert a statement to display its default values or passed in argument values

```
echo "Drinking $tmp $flavor.<br>" ;
```

4 After the function block, add function calls that optionally pass string arguments

```
drink() ; drink( 'iced' ) ; drink( 'cold' , 'lemonade' ) ;
```

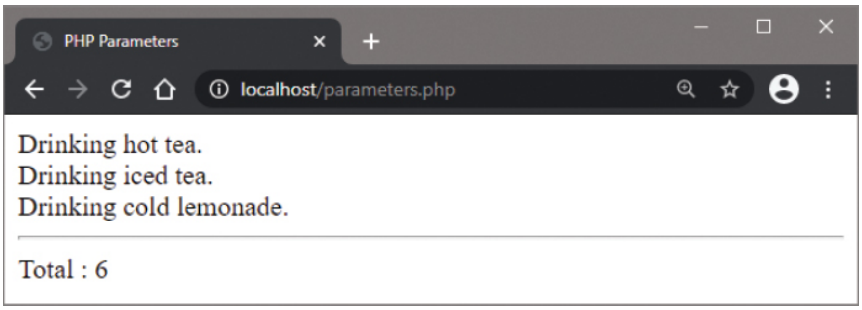
4 Now, define a function that can accept multiple integer arguments and display their total value

```
function add( int ...$numbers )
{
  $total = 0 ;
  foreach( $numbers as $num ) { $total += $num ; }
  echo "<hr>Total : $total" ;
}
```

5 Finally, after the function block add a statement to call the function in the previous step and pass three values

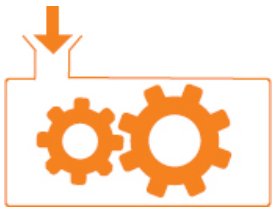
```
add( 1 , 2 , 3 ) ;
?>
```

6 Save the document in your web server's **/htdocs** directory as **parameters.php** then open the page via HTTP to see the argument and default values



You can specify regular positional parameters before a parameter that accepts multiple arguments with the ... splat operator. Only arguments that outnumber the positional elements will be added to the array of the final parameter.

Naming parameters



A function definition will typically include parameters that will require a caller to pass argument values to the function in the same order as the parameters are listed in the function definition. Consequently, these are known as “positional parameters”.

PHP also supports “named parameters” that allow a caller to pass argument values to the function in any order. In this case, the call must specify each parameter name and argument value separated by a colon. The parameter name in the call is the parameter name specified in the function definition – but without the **$** prefix.

```
function function_name ( $arg_1 , $arg_2 )
{
  echo $arg_1 + $arg_2 ;
}

function_name ( arg_1 : ‘Hot’ , arg_2 : ‘Tea’ ) ;     # Outputs Hot Tea
function_name ( arg_2 : ‘Tea’ , arg_1 : ‘Hot’ ) ;     # Outputs Hot Tea
```

When a function definition specifies a default value for a parameter, and the caller does not pass an argument value for that parameter, the default value will be used just as when calling positional parameters.

Named parameters is a great new feature introduced in PHP 8.

The use of named parameters can improve code readability and also allows a caller to pass argument values to specific parameters while assuming default values for other parameters. For example, given a function definition with two parameters that each have specified default values, a positional parameter

call cannot specify an argument value for the second parameter alone. It can't skip the first parameter. A named parameter call, on the other hand, can specify an argument value for the second parameter and assume the default value for the first parameter.

function_name **(** ***arg_2*** **:** ***arg_value*** **) ;**

You can use both named parameters and positional parameters in a function call but all positional parameters must appear before named parameters within the call. Using a positional parameter after a named parameter in a function call will create an error, as the PHP parser cannot correctly assign the argument value.

Additionally, the **...** splat operator can be used in a function definition to collect parameter names and values from a named parameter call, and their names will be preserved.

1. Begin a PHP script by defining a function that provides default string values to its two parameters

```
<?php
function cook( string $prep = 'fried' , string $food = 'rice' )
{
  echo "<li>Serving $prep $food" ;
}
```

named.php

2. Next, add statements to call the function with positional parameters and to call with named parameters

```
cook( 'boiled' , 'egg' ) ;
cook( food : 'cheese' , prep : 'grilled' ) ;
```

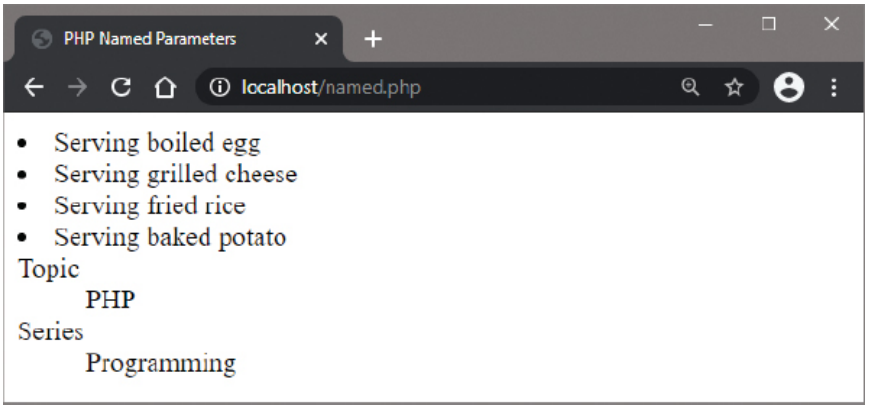
3. Now, add calls to specify an argument value for the second parameter alone, and a call using one positional parameter and one named parameter

```
cook( food : 'rice' ) ; cook( 'baked' , food : 'potato' ) ;
```

4 Then, define a function to collect named parameters and add a call to display passed names and values

```
function register( …$args )
{
  foreach( $args as $name => $value )
  { echo "<dt>".$name."<dd>".$value ; }
}
register( Topic : 'PHP' , Series : 'Programming' ) ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **named.php** then open the page via HTTP to see the argument and default values



Named parameters require all non-optional parameters (those without a specified default value) to be passed an argument value, just as positional parameters do.

Recognizing scope

When creating custom functions it is important to understand how variable accessibility is limited by the variable's "scope":

- Variables created outside a function have "global" scope, but are not normally accessible from within any function block.
- Variables created inside a function have "local" scope so are only accessible from within that particular function block.

As a variable created within a function is not accessible from elsewhere, its name may duplicate the name of a variable outside a function block, or within another function block, without conflict. This allows a global variable to contain a different value to a local variable of the same name.

If you wish to use a global variable inside a function, you must first include a statement with the **global** keyword to declare that the variable name refers to a global variable, like this:

global ***variable-name*** **;**

Global variables in PHP are not like those in other programming languages such as C, where they are automatically available to functions. In PHP, global variables can only be used in functions if first explicitly declared inside the function block using the **global** keyword.

The scope of a variable created within a function can be altered to make it available globally by preceding its declaration with the **global** keyword – but this is bad practice and should be avoided.

Function statements can call other functions to execute their statements and can also call themselves to execute recursively. As with loops, a recursive function must alter the state of a tested condition so it

will not continue forever. This might be a counter value that gets passed in as an argument from an initial function call, then gets incremented on successive recursive calls until it exceeds a value specified in a test. Alternatively, a static local variable could be declared to store an incrementing value, like this:

static *variable-name* ;

Unlike other local variables, which lose their value when the program scope leaves the function, static variables retain their value. This means that the updated value of an incremented value stored in a static local variable is recognized on each function call.

A recursive function might otherwise reference a global variable counter value that gets incremented on successive recursive calls, until it exceeds a value specified in a test.

1. Begin a PHP script by initializing a global variable with an integer and displaying the variable value

```
<?php
$number = 1 ; echo “Global number : $number<br>” ;
```

scope.php

2. Next, define and call a function that initializes a like-named local variable and displays that variable value

```
function show_local()
{
  $number = 100 ; echo “Local number : $number<hr>” ;
}
show_local() ;
```

3. Now, define and call a recursive function that increments the global variable and a static variable on each call

```
function recur()
{
  global $number ;      static $letter = ‘A’ ;
  if( $number < 14 )
```

```
  {
    echo "$number:$letter | " ;
    $number++ ; $letter++ ; recur() ;
  }
}
recur() ;
```

4 Finally, after the function block add a statement to display the modified global variable value

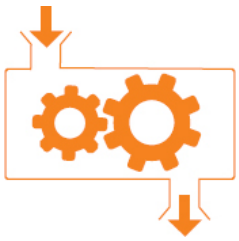
```
echo "<hr>Global number : $number" ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **scope.php** then open the page via HTTP to see the global and local variable values

If you remove the **static** keyword from this function, the **$letter** variable will simply contain **'A'** each time the function is called.

PHP also has a special **$GLOBALS** array in which the value of a global variable can be referenced by its name. In this example you could alternatively use **$GLOBALS['number']** to get the value.

Returning values



A custom PHP function can return a value to the caller by adding a statement using the **return** keyword, like this:

```
function function_name ( )
{
  statement-to-be-executed ;
  statement-to-be-executed ;
  return value ;
}
```

The **return** statement is optional and if omitted, functions will return a **NULL** value to the caller by default.

Functions can only return a single item to the caller, so they cannot return multiple values. You can, however, assign multiple values to elements of an array and have a function return the single array to the caller, then unpack the values.

A **return** statement can include an expression to be evaluated so its single result will be returned to the caller. For example:

```
function square( int $number )
{
  return $number * $number ;
}

echo square( 3 ) ;          # Outputs 9
```

To ensure a function will only return a value of a desired data type, the function declaration can, optionally, specify the return type description such as **int**, **float**, **string**, **bool** or **array**, like this:

```
function function_name ( ) : int
```

```
{
  statement-to-be-executed ;
  statement-to-be-executed ;
  return value ;
}
```

When a return data type is specified, a function that attempts to return an incorrect data type to the caller will result in an error.

1. Begin a PHP script by defining a function that must only return an array of data

```
<?php
function supply() : array
{
  // Statement to be inserted here.
}
```

return.php

2. Next, insert a statement to return an array of four mixed values

```
return array ( 75 , 3.142 , 'Super PHP' , True ) ;
```

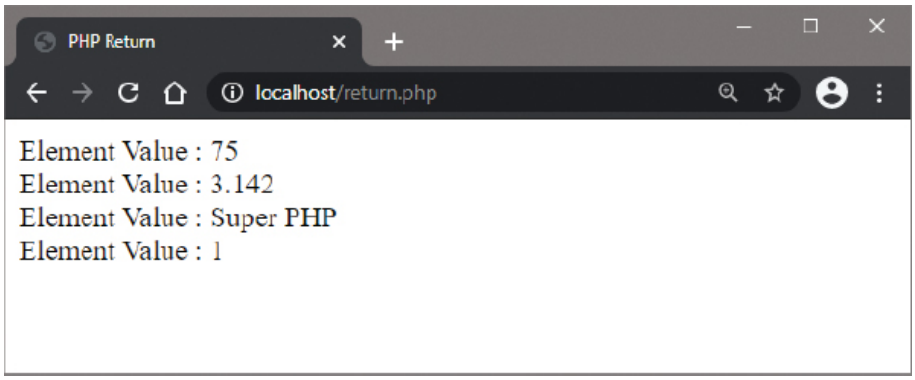
3. After the function, add a statement that calls the function and assigns the returned array values to a variable

```
$array = supply() ;
```

4. Finally, add a loop to display each of the values returned from the function

```
foreach( $array as $data )
{
  echo "Element Value : $data<br>" ;
}
?>
```

5 Save the document in your web server's **/htdocs** directory as **return.php** then open the page via HTTP to see the returned values



In PHP, the Boolean **TRUE** value is represented numerically as 1.

Allowing union types

It is generally preferable to restrict parameters to accept only one specific data type, but sometimes you may want a parameter to allow calls to pass values of different data types. This is possible by specifying the acceptable data types in the function definition as a “union type”. This is pipe-separated list of PHP data types before the parameter name with this syntax:

function *function_name* (*type*|*type*|*type* $*arg*)

Similarly, multiple acceptable data types of the value returned from a function can be specified as a union type with a pipe-separated list in a function definition:

function *function_name* (*type*|*type* $*arg*) : *type*|*type*

Union types support all the PHP data types except for **void**, but does support the special **null** type. This means that a function can return a value of a specific data type or **null**. This is a fairly common requirement known as a “nullable union” . It can be specified as a pipe-separated list or with a shorthand notation:

type*|null** or **?*type

Union types is a great new feature introduced in PHP 8.

The shorthand version can only be used with a single data type, not with a union type, and the **null** type can only be used as part of a union type.

PHP also has a **mixed** pseudo type that allows a parameter or return value to be of any type except for **void**. It is equivalent to a union type that looks like this:

string|int|float|bool|null|array|object|callable|resource

The **mixed** pseudo type cannot be used in union with other types because it already represents all acceptable types. When no data type is explicitly specified for a parameter, PHP assumes the type to be **mixed**. When no data type is explicitly specified for a function's return value, PHP assumes the type to be **mixed** or **void**.

The **callable** type is simply a type hint. It may be specified for a parameter that accepts a callable function.

1 Begin a PHP script by defining a function that can accept an integer or floating-point number as its argument, and can return either an integer or floating-point number

```
<?php
function half( int|float $arg ) : int|float
{
  // Statements to be inserted here.
}
```

union.php

2 Next, insert statements to call another function to describe the argument, and return half of its value

```
echo desc ( '<dt>Argument' , $arg ) ;
return $arg/2 ;
```

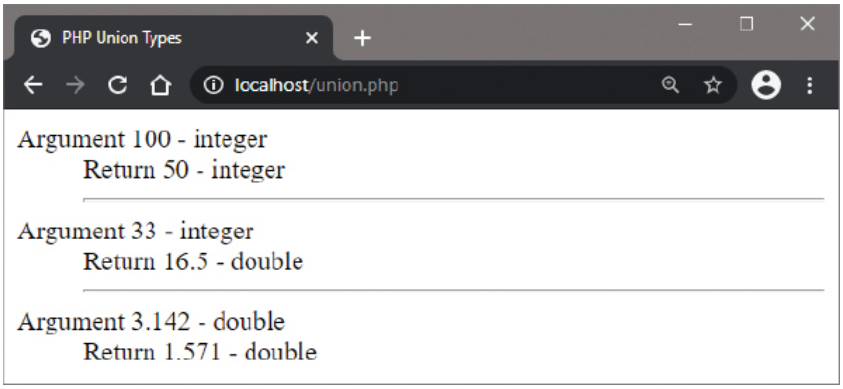
3 Now, add the other function that will return a string describing the numerical argument value it receives

```
function desc( string $str, mixed $val ) : string
{
  return $str.' '.$val.' - '.gettype( $val ) ;
}
```

4 Finally, add statements to call the functions, passing integer and floating-point values

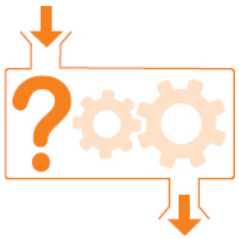
```
echo desc( '<dd>Return', half( 100 ) ).'<hr>' ;
echo desc( '<dd>Return', half( 33 ) ).'<hr>' ;
echo desc( '<dd>Return', half( 3.142 ) ) ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **union.php** then open the page via HTTP to see the integer and floating-point argument and return values



Union types can be used in all places where types can be specified – for parameters, return types, and for class properties in Object Oriented Programming (see Chapter 7).

Calling back



PHP allows you to create “anonymous” functions that have no name specified in the function definition. Like other functions, these can accept arguments if parameters are specified and they can optionally return a value when called. Unlike other function definitions, which are code constructs, an anonymous function definition is an expression. This means that each anonymous function definition must be terminated by a semicolon, like this:

```
function ( )
{
  statement-to-be-executed ;
  statement-to-be-executed ;
  return value ;
} ;
```

Don’t forget to add a terminating **;** semicolon after each anonymous function definition.

On its own an anonymous function is of little use as, without a name, it cannot be called, but it can be useful in other ways:

- Assigned to a variable, an anonymous function can be called using the variable name.
- Passed as a callable argument to another function, an anonymous function can be called later as a “callback”.
- Returned from a function, an anonymous function retains access to the returning function’s variables in “closure”.

By assigning an anonymous function to a variable you do, in effect, give the function a name and can call it and pass it arguments as you would any other function – using the variable name. Similarly, when you pass an anonymous function to another function it can be called back and passed arguments as you would any other function – using the parameter name. The PHP **use** keyword and **&** ampersand prefix allow an anonymous function to reference a local variable in the returning function.

1 Begin a PHP script by assigning to a variable an anonymous function that accepts a single argument

```
<?php
$hello = function ( $user ) { echo “Hello $user!<br>” ; } ;
```

callback.php

2 Next, add a statement that calls the anonymous function and passes a single argument

```
$hello( ‘Mike’ ) ;
```

3 Now, define a regular function that accepts a single callable argument and calls that function passing a string

```
function greet( callable $anon )
{
  $anon( ‘Carole Anne’ ) ;
}
```

4 Add a statement to call the function, defined in the previous step, passing the anonymous function that was assigned to a variable in the first step

```
greet( $hello ) ;
```

5 Next, define another regular function that has one local variable and returns an anonymous function that references the local variable value

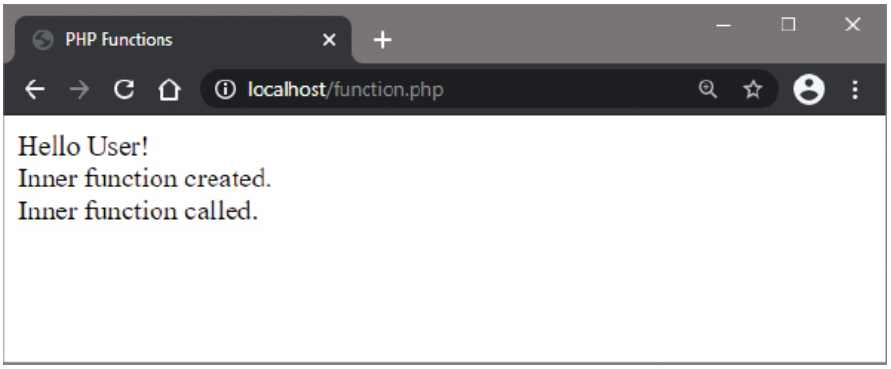
```
function meet() : callable
{
  $time = ‘morning’ ;
```

```
  return function( $name ) use( &$time )
  { return "Good $time, $name!" ; } ;
}
```

6 Assign to a variable the function defined in the previous step then call that function, passing a string argument

```
$meeting = meet() ;
echo $meeting( 'Susan' ) ;
?>
```

7 Save the document in your web server's **/htdocs** directory as **callback.php** then open the page via HTTP to see the returned values from anonymous functions



Remember to include the **&** ampersand prefix to create a reference to the variable value.

Notice that the final function call gets the **'morning'** string from the parent scope.

Summary

- Custom functions allow PHP scripts to be modularized into sections of code that are easier to understand and maintain.
- Function names must begin with a letter or underscore character and are not case-sensitive.
- A function is defined by stating its name after the **function** keyword, then parentheses and braces containing statements.
- A function accepting arguments is defined by including a comma-separated list of parameters within the parentheses.
- Optionally, a function definition can specify the type of data to be passed from the caller before the parameter name.
- Argument data is normally passed by value but can be passed by reference if the parameter name is given an **&** prefix.
- Default argument values can optionally be specified in a function definition by assignment to the parameter name.
- The **...** splat operator allows multiple arguments to be passed to a parameter in order to create an array.
- Named parameters allow a caller to pass argument values to the function in any order.
- Variables created outside a function have global scope, but are not accessible from within a function block unless it includes a declaration using the **global** keyword.
- Variables created inside a function have local scope so are only accessible from within that particular function block.
- Local variables lose their value when the program scope leaves the function, but **static** variables retain their value.
- The **return** keyword returns a single item to the caller and the function definition can optionally specify the return data type.
- A union type is a pipe-separated list of PHP data types that allow various types of arguments and returns.

- The **use** keyword and **&** ampersand prefix allow an anonymous function to reference a local variable in the returning function.